

Prove that consistency implies admissibility and give an example of an admissible heuristic that is not consistent.
A heuristic is admissible if it never overestimates true cost to the goal.

A heuristic is consistent if the estimated cost from a node to the goal is no greater than the cost of moving to a neighbour plus the neighbour's estimated cost to the goal.

Proof:

Assume that $h(t) = 0$ for any goal.

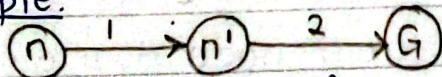
Let predecessor of goal node t , be n .

If the heuristic function is consistent, then:

$$\begin{aligned} h(n) &\leq c(n, t) + h(t) = c(n, t) + 0 \\ &= c(n, t) \end{aligned}$$

and hence h behaves admissibly here.

Example:



Given $h(n) = 3$, $h(n') = 1$

Admissibility: $h(n) \leq h^*(n)$

$$h^*(n) = 1 + 2 = 3$$

$h(n) = 3 \leq h^*(n) = 3$ hence admissible

$$h(n') \leq h^*(n') = 2$$

$h(n') = 1 \leq 2$ hence admissible

Consistency:

$$h(n) \leq c(n, n') + h(n')$$

$$\begin{aligned} c(n, n') + h(n') &= 1 + 1 \\ &= 2 \end{aligned}$$

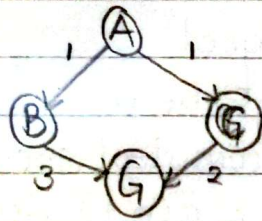
But $h(n) = 3$

and $h(n) \leq 2$ is false therefore inconsistent.

Show that an inadmissible heuristic can lead to A^* returning a suboptimal example, by giving an example search tree and heuristic where this occurs. Aim for a small a tree as possible, and explain your example clearly.

If an inadmissible heuristic was used, it is possible that the algorithm would not reach the optimal solution because of an overestimation in the evaluation function.

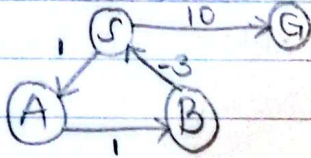
Example:



A is the starting node and G the goal node. Let the heuristics be: $h(B) = 3$ and $h(C) = 4$. This heuristic function h will not be admissible because $h(C) = 4 > c(C, G) = 2$.

If the A^* algorithm starts from node A, it will then select the node B for the purpose of expansion and after will proceed to node G. The path will be $A \rightarrow B \rightarrow G$ with cost 4 instead $A \rightarrow C \rightarrow G$ with cost 3. If the heuristic function had been admissible $A \rightarrow B$ could be chosen for the next node to expand, but after that, the A^* would select $A \rightarrow C$ and in the end, it would end up with $A \rightarrow C \rightarrow G$.

Give a detailed example (by drawing the state and labelling the edges with costs) where negative edge costs could lead to the following paradoxical result: a finite state space with an optimal solution that does not reach the goal in finite time.



Assuming S is the start and G is the goal. Cumulative costs of different paths:

$S \rightarrow G$ which is direct is 10.

$S \rightarrow A \rightarrow B \rightarrow S$ which is a cycle is -1.

Multiple cycles then goal where k is each iterate:

$$k \cdot (-1) + 10 = 10 - k$$

The cycle $S \rightarrow A \rightarrow B \rightarrow S$ has a negative cost of -1. By traversing this cycle before going to the goal, we reduce the cost. After
As $k \rightarrow \infty$, total cost $\rightarrow -\infty$ and therefore the optimal solution would traverse the cycle infinitely.

Traversing the cycle means finding cheaper costs and that means improving the total cost, therefore, never settling on the goal G as the option of finding a cheaper cost is there.

Suppose you are asked to apply search to a symbol problem. You are told that the solution is a sequence of at most five symbols, but the symbols are drawn from an alphabet which contains thousands of symbols. Formulate this problem as a search problem. If depth-first and breadth-first were your own choices, which one would you apply? Explain. How might you change the algorithm?

Search problem:

Problem: Draw a sequence of five symbols from an alphabet containing thousands of symbols.

Initial state: The empty tuple

Goal state: The state is a valid solution sequence. The formulation could be $1 \leq \text{len}(\text{state}) \leq 5$

Action: Appending a symbol to the tuple if the length of the tuple is not equal to 5.

Given the choice between DFS and BFS, I would apply depth first search as it is memory efficient compared to BFS which has a high memory usage as it stores all nodes at the current level.

I might change the algorithm by adding the possibility of excluding impossible symbol choices early

Modify iterative deepening search so that it finds the lowest cost, as opposed to shortest path. What information do you need? Prove that your algorithm finds the optimal solution.

ID_S combines DFS's space-efficiency and BFS's fast search (that is for nodes closer to the root). ID_S repeatedly runs Depth-Limited Search with increasing depth limits. Modify ID_S to find the lowest-cost path, we replace the depth limit with a cost limit and introduce a heuristic.

To compute the lowest-cost path you must know the cost of the path ~~to~~ from the start to node n , a heuristic estimate of the cost from n to the goal and the total estimated cost.

Proof that the algorithm finds the optimal solution:

The first iteration explores all paths with cost under the bound. If the goal is found, it must have cost $\leq$ bound.

Each iteration increases the bound to the smallest f that exceeded the previous bound. So we gradually explore paths in increasing order of total estimated cost f .

No goal state with smaller true cost is pruned, because $f = g + h \leq$ true cost for all nodes.

The first goal found must have the smallest true path cost, since all cheaper paths have already been explored.